

Interfaces. Caso Zoo virtual

Índice

Zoológico virtual	2
Planteo del problema	2
Una primera aproximación	3
Una solución posible: herencia	4
Una solución mejorada: interfaces.....	7

Zoológico virtual

Planteo del problema

Se quiere modelar un zoológico virtual. Como todo zoológico, se quiere simular todo tipo de animales. Cada animal tendrá un identificador, un nombre y una fecha de ingreso al sistema (edad).

Al igual que en otros zoológicos, se deberá poder representar animales mamíferos, otros ovíparos, habrá peces de agua salada y dulce, reptiles e insectos.

Como el zoo recién comienza a funcionar, solo se cuentan con algunas especies de mamíferos. Por el momento hay felinos, caninos, un par de elefantes, una pareja de hipopótamos y tres jirafas. Afortunadamente, la hembra de la pareja de hipopótamos está preñada y el zoo tendrá su primera cría en cautiverio virtual! De todas formas, todos los mamíferos deben tener la posibilidad de tener crías, así que se debe modelar que cada mamífero guarde la cantidad de crías posible. Por supuesto, los mamíferos deberán ser capaces de parir y de amamantar a sus crías. Por el momento, dentro del recinto de los felinos tenemos un león, un tigre, y un puma. Dentro del recinto de los caninos sabemos que habrá una manada de lobos, una pareja de zorros y se simulará una pareja de todas las razas de perros del mundo!

Dentro del recinto de ovíparos, tenemos principalmente aves. El zoo está esperando que lleguen las tortugas de mar virtuales, pero el equipo de desarrolladores todavía está testeando el código. Los dragones de Cómodo están en etapa de análisis. Todas las aves, además de ser capaces de volar (obvio), como todos los ovíparos, deben tener la capacidad de poner huevos y romper el cascaron al nacer.

La zona del acuario todavía no está en condiciones de recibir peces, pero sí está listo el tanque transparente más grande con dos delfines.

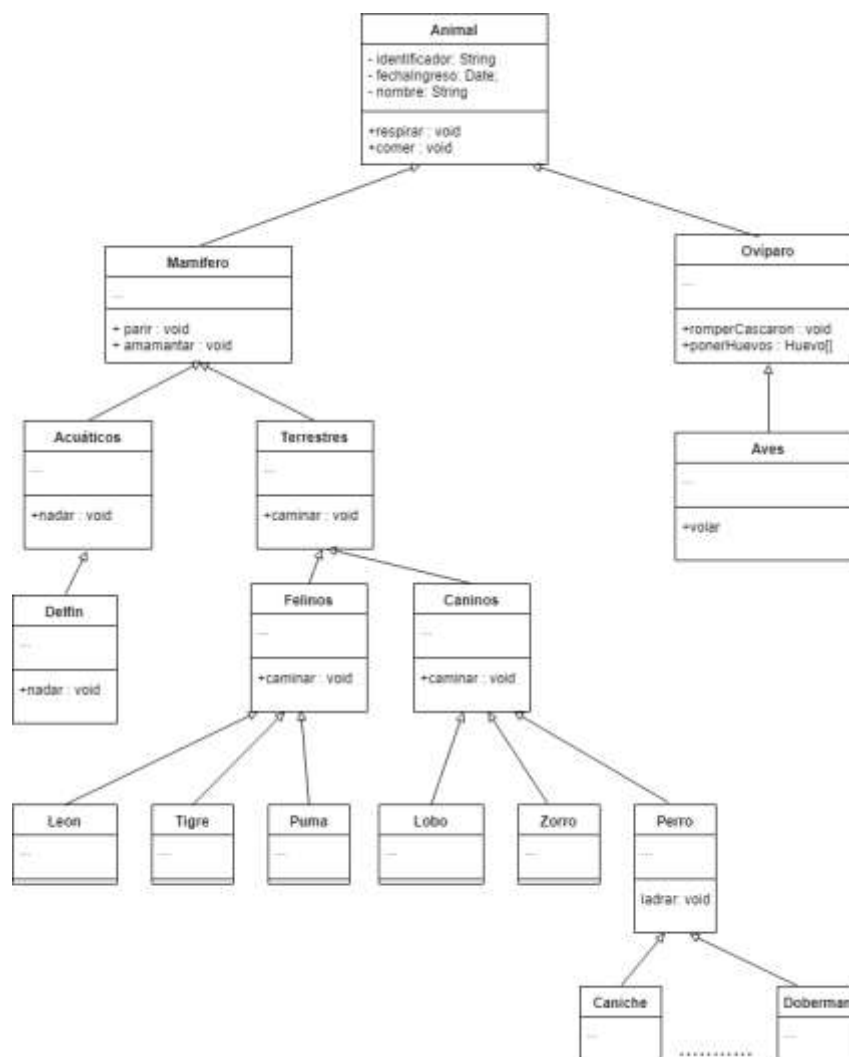
Recientemente, se ha unido al zoo (se deployó en producción hace unos días) un animal exótico: el ornitorrinco. El ornitorrinco es un animal muy raro, dado que, si bien es un

mamífero, la hembra pone huevos. Luego, cuando salen las crías del cascarón, la madre se dispone a amamantarlas.

Es muy importante que nuestro zoo virtual pueda soportar animales tan extraños como los ornitorrincos.

Una primera aproximación

Habiendo visto el concepto de *herencia*, lo primero que podemos hacer es generar una jerarquía de animales virtuales, con el objetivo de reutilizar código. Mamíferos, ovíparos, etc., todos respiran, comen, se mueven de forma similar. Podemos hacer una jerarquía con métodos en común y sobrescribir las operaciones que necesitemos en cada caso (ver diagrama siguiente. Algunos detalles se omitieron por simplicidad).



Hasta este punto, todo parece encajar, el problema será incorporar al ornitorrinco, sabiendo que debe tener la posibilidad de poner huevos y también amamantar a sus crías. Debe ocurrir algo similar con romperCascarón. Por ahora centremos la atención en poner huevos y amamantar. Supongamos que tenemos un ornitorrinco:

Ornitorrinco

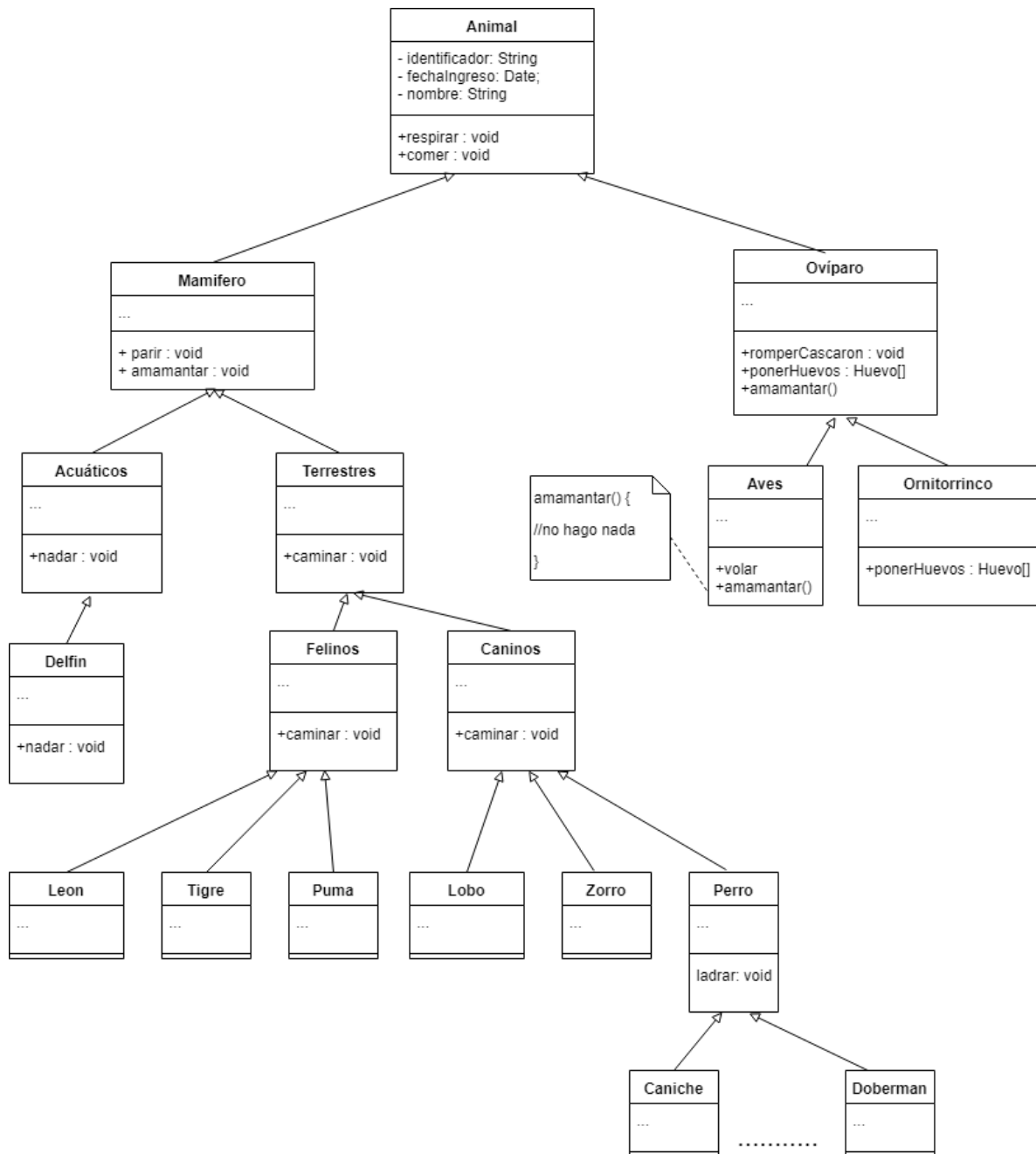
+ponerHuevos : Huevo [] +amamantar : void

Podríamos incluir la clase ornitorrinco en nuestro esquema actual, con todas las operaciones necesarias. Aunque de esta manera perderíamos todo el código que tenemos para todos los animales, mamíferos, ovíparos, etc.

Tenemos también la restricción de Java sobre la herencia. En Java no hay herencia múltiple. Por ello, deberíamos resolver el problema de alguna otra manera.

Una solución posible: herencia

Podríamos pasar el método amamantar() a la clase ovíparo, que tendría cierto sentido, sabiendo que hay algunos (muy, muy pocos –familia de “monotremas”– 1 especie de ornitorrinco y 4 especies de Equidnas) ovíparos que amamantan, no sería tan grave. Pero eso daría lugar a que el método amamantar() deba permanecer “sin implementar” en las clases hijas de ovíparo, dejando la implementación vacía o tirando algún error cuando se quieren ejecutar. Veamos:



Nuevamente, no es tan grave, pero de alguna forma estamos “contaminando” todas las aves con una operación o comportamiento que, en realidad, no deberían tener. Pasaría algo similar si ponemos “ponerHuevos” del lado de los mamíferos. Por solo 5 especies en el mundo, estamos agregando a todos los mamíferos una operación que, sabemos, no realizan.

Por lo tanto, la solución no siempre es poner en algún nivel “común” una operación para que solucione “algunos” de los casos necesarios (y así, complique otros). Si lo que queremos es tener comportamiento polimórfico entre mamíferos, ovíparos y “raros” (en este caso el ornitorrinco) de forma tal que podamos hacer:

```
Oviparo o = new Ornitorrinco()
o.comer(); //todos los oviparos comen

//bien, los ovíparos que son ornitorrincos amamantan
o.amamantar();
...
...
o = new Loro();
o.comer(); //todos los oviparos comen, loros u ornitorrincos
o.amamantar(); // algunos ovíparos amamantan, pero un Loro no
```

Si tenemos este mismo código, y sin complicarnos, quisiéramos usar instanceof, estaría bien. Pero estamos limitando seriamente el polimorfismo, dado que no solo dependemos de ciertas instancias, sino que estamos agregando una condición:

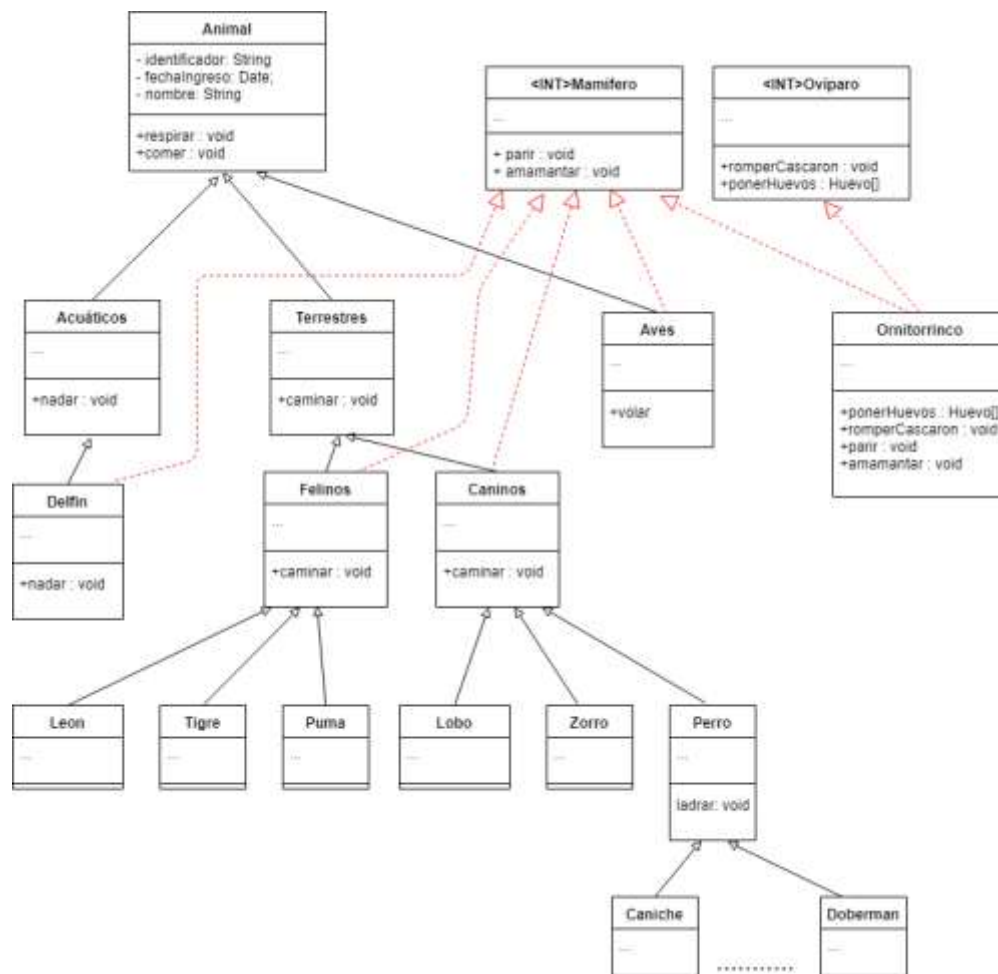
```
if (o instanceof Ornitorrinco) {
    ((Ornitorrinco)o).amamantar();
}
```

Lo que implica que si luego agregamos otra especie de monotrema (ver más arriba) tenemos que agregar otro if. Si luego hay que sacarlo, tendríamos que volver a modificar el código. Esto atenta directamente contra el propósito del polimorfismo, la reutilización de código y el diseño modular que idealmente debe tener una pieza de software. Agregar o quitar especies no debería implicar tocar código, mucho menos lógica, que requeriría tests para comprobar si es correcta.

Una solución mejorada: interfaces

Si bien no tenemos herencia múltiple, ¿qué mecanismo podríamos usar para poder hacer que un ornitorrinco sea mamífero y ovíparo al mismo tiempo? ¿Qué herramientas tenemos para lograr polimorfismo y a la vez solucionar este problema? Las interfaces.

Si tuviéramos una interfaz ovíparo y una interfaz mamífero podríamos dar los comportamientos de amamantar() y de ponerHuevos() (y romperCascaron() y parir()) para el caso) en estas interfaces y, de esa manera, dar selectivamente estos comportamientos a las clases que lo necesiten. Entonces, no necesariamente todos los animales son ovíparos o mamíferos. A su vez, no tendremos mamíferos que ponen huevos u ovíparos que amamantan como en el caso anterior. Si bien no es la mejor de las soluciones es una solución mucho mejor. Ya veremos, en un apunte posterior, otras formas de resolver estos tipos de problemas. (Las flechas están en rojo para resaltar).



De esta forma, podremos hacer:

```
Oviparo o = new Ornitorrinco()
o.comer(); //todos los oviparos comen

//bien, los ovíparos que son ornitorrincos amamantan
((Mamifero)o).amamantar();
...
...
o = new Loro();
o.comer(); //todos los oviparos comen, loros u ornitorrincos
//esto ya no es valido
//o.amamantar(); // pero un Loro no da de amamantar
```

Si revisamos el código, veremos una operación de casteo. La mejora respecto del caso anterior es que, para dar de amamantar, si bien vamos a tener que castear, y siempre para eso, preguntar instanceof, ya no hay que hacerlo para cada especie en particular.

```
if(o instanceof Mamifero) {
    ((Ornitorrinco)o).amamantar();
}
```

Lo que implica que si luego agregamos otra especie de monotrema, ya no tenemos que agregar otro if, porque este será mamífero (y ovíparo). Si luego hay que sacarlo, no habrá que modificar el código. Mantuvimos el polimorfismo entre los mamíferos 8 y entre los ovíparos).

Nuevamente, si bien esta no es una solución ideal, es mejor que la primera. Veremos en un apunte posterior, con un problema más complejo, cómo solucionar este tipo de problemas de forma “ideal”.